



TED UNIVERSITY

Faculty of Engineering

CMPE 492/SENG492 Senior Project II

LLM Valley Final Report
15.05.26

by

Yusuf Sadi Pesen
Deniz Şeker
Egemen Asım Ersoy
Reyhan Sena Çimen

Supervisor:

Dr. Eren ULU

Jury Members:

Dr. Ulaş GÜLEÇ
Dr. Ayşenur BİRTÜRK

Table of Contents

1- Introduction and Scope	3
1.1 Scope of This Final Report	3
1.2 Project Goals and Final Direction.....	3
1.3 Final Project Status	3
1.4 Impact of the Engineering Solution	4
1.4.1 Societal Impact	4
1.4.2 Economic Impact	4
1.4.3 Environmental Impact	4
1.4.4 Global and Engineering Context.....	4
1.5 Contemporary Issues Related to the Project	5
2- Architecture and Design	5
2.1 System-Level Architecture	6
2.2 Main Interaction Flow	7
2.3 Gameplay Loop and System Integration.....	7
2.4 Persistence, Reliability, and Safety	8
2.5 Repository Information	8
2.6 Deviation from Low Level Design.....	9
2.7 Limitations of the Current System	10
3- Test Results.....	11
3.1 Functional Tests.....	11
3.2 Interface Tests.....	12
3.3 Validation Tests	12
3.4 Test Assessment	13
4- Tools and Technologies Used	14
4.1 Use of Background and Reference Resources.....	14
5- Learned Lessons	15
6- REFERENCES.....	17

RELATED DOCUMENTS

Name	URL
Low Level Design	https://drive.google.com/file/d/1IYTABR_hXSW4mH19BFncFa4tTala4m8k/view?usp=sharing
Test Plan	https://drive.google.com/file/d/1qvmQPSXROSIO6SiUYAT5ggUzjQ5rjXXq/view?usp=sharing
Project Analysis Report	https://drive.google.com/file/d/1iUy-j6eruHpVj_qtyBSuAxHp2_bl50yz/view?usp=sharing
Project Specifications Report	https://drive.google.com/file/d/1NDD0vGwtczSk91MV4W91MkBJuZKVDVdm/view?usp=sharing
High Level Design Report	https://drive.google.com/file/d/1HzcaziJZhS9BSP4uBkztiZBGLhXF92Wk/view?usp=sharing
Project Proposal	https://drive.google.com/file/d/1RvbXkLjcdAaJc4TCI63tESMzvVnDYB6n/view?usp=sharing
Website	https://llmvalley.vercel.app/
Github	https://github.com/egemenasim/LLMValley

CHANGE HISTORY

Date	Changed by	Change Description
10.05.2026	Yusuf Sadi Pesen	Game design-related parts of the report were prepared, including the general gameplay structure, simulation loop, and project direction.
11.05.2026	Deniz Şeker	Architecture and design section, system-level structure, gameplay loop, and deviation from low-level design were written.
12.05.2026	Egemen Asım Ersoy	NPC interaction and LLM integration-related sections were prepared, including dynamic dialogue flow, OpenRouter API usage, and the final AI-supported demo status.
13.05.2026	Reyhan Sena Çimen	Test results and test assessment sections were completed, including functional, interface, and validation test summaries.

1- Introduction and Scope

LLM Valley is a narrative-driven life simulation demo developed to explore the integration of Large Language Models (LLMs) into real-time Non-Player Character (NPC) interaction systems. Traditional life simulation games usually depend on static dialogue trees. In LLM Valley, this structure is replaced with free-text player input and context-aware NPC responses generated through an external LLM service. The project studies whether this approach can make NPC interaction feel more natural while still keeping the overall game loop understandable and technically manageable.

The game itself primarily serves as an interactive environment for evaluating the proposed LLM-driven NPC interaction system and it is designed as a small but complete playable demo. The player can move inside a cozy village environment, interact with NPCs, perform farming activities, trade items, and advance the in-game day by sleeping. The project also includes relationship-related statistics such as Love, Trust, Friendship, and Mental, which connect dialogue outcomes to gameplay progression. These systems were defined in the earlier proposal, analysis, specification, high-level design, and low-level design documents.

1.1 Scope of This Final Report

This Final Report presents the final state of the LLM Valley project. It explains the current system architecture, the completed implementation, the main design decisions, and the results obtained from the testing process. The report also discusses the tools and technologies used during development, the lessons learned throughout the project, and the broader economic, environmental, and societal impact of the proposed system.

1.2 Project Goals and Final Direction

- Develop a working LLM-driven NPC dialogue system that accepts natural player input and returns dynamic NPC responses.
- Build a minimal but complete life simulation loop including farming, trading, relationship building, and day progression.
- Keep the core game logic separate from the LLM service layer so that the AI provider can be changed without redesigning the whole codebase.
- Use lightweight data handling and local JSON persistence rather than a cloud database, since the project is a single-player demo.
- Maintain stable gameplay flow by using asynchronous API communication and fallback behavior during API errors or timeouts.
- Demonstrate the feasibility of integrating cloud-based generative AI systems into a playable real-time game prototype.

1.3 Final Project Status

LLM Valley was successfully completed in Unity as a playable demo version of the proposed narrative-driven life simulation game. The final demo includes the main gameplay structure planned in the earlier documents, such as player movement in the village environment, NPC interaction, dynamic dialogue generation, and the overall simulation flow designed for the project.

For the LLM-powered NPC dialogue system, the project uses the OpenRouter API with the Claude Haiku 4.5 model. Through this integration, the player can write free-text messages to

NPCs, and the game can generate contextually relevant responses during gameplay. This allowed us to demonstrate the main technical goal of the project: replacing fixed dialogue trees with real-time, LLM-supported NPC conversations inside an actual game environment. The demo version was tested and was able to run successfully with the implemented LLM dialogue flow. Player inputs were sent to the model through the API, responses were received and displayed in the game interface, and the interaction loop worked as expected during the demonstration process. The completed tests showed that the core communication pipeline between the Unity game client and the external LLM service functions correctly and supports the intended gameplay experience.

Overall, the final state of the project fulfills the primary objective defined at the beginning of the semester: developing a functional proof-of-concept system that demonstrates how Large Language Models can be integrated into real-time NPC interaction systems. The final prototype successfully establishes stable communication between the Unity-based game client and the external LLM service, enabling dynamic and context-aware NPC conversations within the gameplay environment.

1.4 Impact of the Engineering Solution

1.4.1 Societal Impact

LLM Valley demonstrates how dynamic dialogue systems can support more flexible player-NPC interaction. Instead of limiting the player to predefined dialogue options, the system allows natural language communication supported by an external LLM service. This approach may contribute to more immersive gameplay experiences and also has potential relevance for accessibility-focused applications, educational simulations, and interactive training environments. Although the project is implemented as a small-scale prototype, it provides a practical example of how conversational AI can be integrated into interactive digital systems.

1.4.2 Economic Impact

From a development perspective, a dynamic dialogue pipeline may reduce some of the manual writing effort required for large static dialogue trees. At the same time, the project documents correctly recognize that API-based LLM usage can create cost concerns. Therefore, the design keeps the system modular and lightweight, and the project proposal already mentions the need to keep API use within a planned budget. The current demo uses a limited scope rather than a large-scale production deployment.

1.4.3 Environmental Impact

LLM Valley is a fully digital software project and does not require physical manufacturing processes. The system is designed to run on standard consumer hardware without requiring local high-performance AI inference, since text generation is handled through external LLM APIs. This approach helps keep the local hardware requirements relatively lightweight while maintaining access to advanced language model capabilities.

1.4.4 Global and Engineering Context

The project belongs to a growing engineering direction where generative AI is connected to real-time applications. LLM Valley contributes to this area at a student project scale by showing how a life simulation game can combine AI dialogue, state-based control, game statistics, and responsive UI behavior inside one playable architecture. The project also raises important design questions about reliability, response latency, prompt control, and content safety, which are relevant beyond a single game prototype.

1.5 Contemporary Issues Related to the Project

- Response latency in real-time generative systems: NPC replies must arrive quickly enough to avoid interrupting the player experience.
- Content safety and prompt control: AI-generated dialogue should avoid harmful, discriminatory, or inappropriate outputs.
- Data privacy: player text must be handled carefully and should not be stored unnecessarily.
- Consistency of NPC behavior: dynamic text should still remain compatible with the NPC persona, the current relationship state, and the world context.
- Maintainability of AI-supported software: switching model providers or adjusting prompts should not require a full rewrite of game logic.

2- Architecture and Design

The current design of LLM Valley follows a client-server model integrated with Unity's component-based development approach. The game client handles rendering, player movement, interface updates, and local simulation logic. The LLM service handles text generation for NPC conversations through a remote API layer. This separation is important because it allows the game to remain playable on mid-range hardware while still offering AI-generated responses.

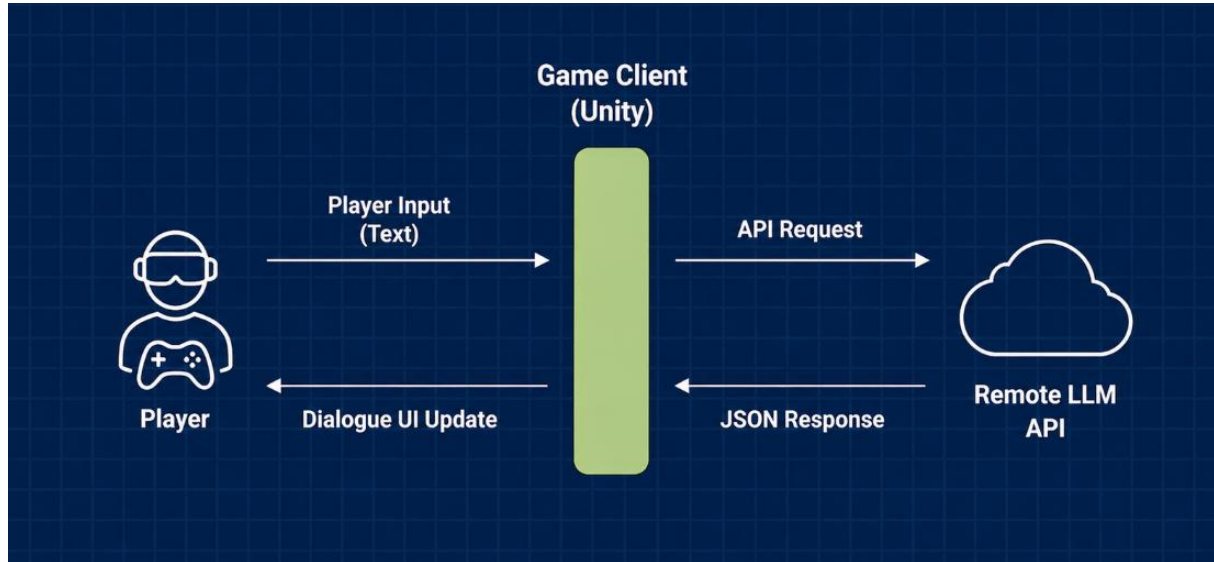


Figure 1. Client-server interaction flow used in LLM Valley.

2.1 System-Level Architecture

The architecture can be summarized through three connected layers: the Game Client, the AI Integration Module, and the Simulation Logic. The Game Client is responsible for the player-facing experience. The AI Integration Module formats player input into prompts, sends requests through HTTPS, receives the generated response, and forwards usable dialogue content back to the game. The Simulation Logic controls farming progression, trading behavior, relationship statistics, and persistence.

Subsystem	Main Responsibility	Actual / Better Example Classes
Core Game & Persistence	Scene transitions, save/load systems, global state management, day/time progression	SaveManager, SceneTransitionManager, GameStateStore, CalendarSystem, CalendarDate, CalendarDatabase
AI & Dialogue / NPC	NPC conversations, LLM/OpenRouter API requests, prompt generation, conversation history, relationship stats	NPCChatAgent, NPCOpenRouterClient, NPCChatUIManager, NPCConversationStore, NPCPersona, NPCRelationshipStats, NPCChatMessage, NPCRelationshipEvaluation
Player & Interaction	Player movement, input locking, inventory, currency, tool usage, interactions	PlayerMove, Move, PlayerInventory, PlayerWallet, PlayerToolController, PlayerPersistence, Interactable, Door, Sleepable, Chest
Items & Inventory	Item definitions, stack logic, collectible item flow, key item validation	ItemData, KeyItemData, ItemStack, ItemType, CollectibleItem, IItemCollector, IInventoryKeyProvider
Farming & Environment	Soil tilling/watering, planting, crop growth, farm tile save data	FarmableArea, Plantable, BasicPlant, FarmableAreaState, FarmTileData, LocalFarmTileTable, GlobalFarmTileDataTable, FarmSaveEventBus

Table 3. LLM Valley Subsystem Summary

2.2 Main Interaction Flow

The central interaction flow begins when the player approaches an NPC and enters a free-text message. The dialogue system combines this input with relevant context such as the NPC persona and game state. The message is sent to the selected LLM API. While waiting for the response, asynchronous operations are used so that the Unity interface does not freeze. After the response arrives, it is displayed in the dialogue UI and may trigger stat-related updates based on the project's planned keyword-driven logic.

1. Player starts an interaction with an NPC.
2. The player enters a free-text sentence using the dialogue UI.
3. The AI module prepares a structured prompt using context and persona information.
4. The LLM API receives the request and returns a response.
5. The game shows the response through the Dialogue UI.
6. Relationship-related values and future interaction conditions may be updated according to the defined stat logic.

2.3 Gameplay Loop and System Integration

LLM Valley is not designed as an isolated conversation demo. Instead, the project integrates the LLM-based NPC interaction system into a small but complete life simulation gameplay loop. The player can move freely inside the village environment, plant and harvest crops, manage inventory items, trade through the shop system, interact with NPCs, and advance the in-game day through sleeping mechanics. These gameplay systems provide environmental and gameplay context for the dialogue system, allowing NPC interactions to exist as part of an active game world rather than as standalone text exchanges.

The different subsystems continuously interact with each other during gameplay. Farming actions affect inventory contents, inventory items influence trading operations, and day progression updates environmental states such as crop growth and NPC-related events. Relationship statistics including Love, Trust, Friendship, and Mental are also connected to the dialogue flow and are updated based on player interaction outcomes. This creates a gameplay structure in which dialogue, progression systems, and world-state management operate together within the same architecture.

The integration of these systems was important for evaluating whether real-time LLM-supported dialogue could function reliably inside an interactive gameplay environment. Rather than generating responses independently from gameplay, the dialogue system operates together with persistence management, UI updates, event-driven state transitions, and player progression systems throughout the gameplay loop.



Figure 2. Example gameplay demonstration

2.4 Persistence, Reliability, and Safety

The game uses a lightweight local save approach built on PlayerPrefs and JSON serialization, which is appropriate for LLM Valley’s single-player demo structure. Rather than relying on an external database, the game persists the key state needed to continue play smoothly between sessions, including the calendar state, inventory contents, selected hotbar slot, player position and rotation, gold amount, and scene information through SaveManager. In addition, NPC conversations and relationship progress are stored locally in JSON files so dialogue history and relationship-driven behavior can be resumed without requiring a remote backend.

Reliability was also a major consideration in the dialogue flow. The conversation system is split across agent and router which makes it easier to manage API requests, loading states, and error handling independently. If the OpenRouter request fails, returns an empty response, or cannot be parsed, the system surfaces an error message instead of breaking the interaction flow. The chat logic also separates normal conversation handling from relationship evaluation, so stat updates can be processed in a controlled way after batches of player messages.

Safety was considered carefully because the game accepts free-text user input and forwards it to an AI model. The project keeps player conversation data local rather than storing it on external servers, and the relationship evaluation system is designed to reduce uncontrolled stat changes by using structured metadata and explicit prompts. This helps limit harmful or off-topic outputs, while still allowing the NPCs to respond naturally within their intended character roles.

2.5 Repository Information

GitHub repository URL(s): <https://github.com/egemenasim/LLMValley>

2.6 Deviation from Low Level Design

The final implementation remained mostly consistent with the Low-Level Design Report. The overall Unity-based architecture, gameplay structure, local persistence approach, and LLM-supported NPC interaction flow were preserved. The most visible change was made in the selected LLM service. While the design documents initially considered DeepSeek or Gemini APIs, the final demo was implemented using the OpenRouter API with the Claude Haiku 4.5 model. This change did not require a redesign of the system, since the AI service layer had already been planned in a modular way.

Design Area	Low-Level Design Direction	Final Report Assessment
LLM Provider	DeepSeek or Gemini APIs were considered as the external LLM service.	The final demo uses OpenRouter API with Claude Haiku 4.5. This is a provider-level change, but it remains consistent with the modular LLM integration approach.
Game Engine and Core Structure	Unity-based component architecture with separated gameplay, AI dialogue, UI, and simulation systems.	The project was completed in Unity, and the core architectural direction was preserved in the implemented demo.
Dialogue System	Free-text player input is sent to an external LLM service and displayed as NPC dialogue.	This feature was successfully implemented and tested in the playable demo. NPCs can produce dynamic replies during gameplay.
Reliability of LLM Flow	API failures or delays should be handled without breaking the overall game flow.	The implemented dialogue system was tested successfully during demo use, and the core API request-response interaction worked as expected.
Persistence	Local JSON-based save/load mechanism was planned for demo-scale data persistence.	The local persistence direction was kept as part of the project design, consistent with the single-player demo scope.
Overall Scope	A functional proof-of-concept life simulation demo demonstrating LLM-powered NPC interaction.	The final project achieved this central goal through a playable Unity demo with working LLM-based NPC conversations.

Table 4. Summary of Alignment with the Low-Level Design

2.7 Limitations of the Current System

As a proof-of-concept prototype, LLM Valley focuses primarily on demonstrating the integration of real-time LLM-supported NPC interaction within a playable gameplay environment. For this reason, some systems were intentionally implemented in a simplified and lightweight form in order to prioritize stability, modularity, and successful AI integration during development.

The current version uses a relatively compact gameplay scope and a limited conversation context window for NPC interactions. While this structure was sufficient for demonstrating the intended interaction flow, future iterations could expand the system with more advanced long-term NPC memory, larger world interactions, and additional gameplay mechanics. Similarly, the modular architecture of the project allows the dialogue system, AI provider, and gameplay subsystems to be further extended and improved without requiring major architectural redesigns.

3- Test Results

This section summarizes the main test results obtained from the completed LLM Valley demo. The tests focused on the functional behavior of the game, the interaction between the Unity client and the LLM service, and the overall validation of the project goals. The results show that the core systems required for the playable demo operated successfully during testing.

3.1 Functional Tests

Test ID	Feature	Expected / Reported Verification Target	Result
FT-01	Player Movement	The player should be able to move through the village map using the implemented control system without blocking the normal gameplay flow.	Passed
FT-02	NPC Interaction	The player should be able to approach an NPC and start a dialogue interaction through the game interface.	Passed
FT-03	Free-Text Dialogue Input	The game should accept a free-text message written by the player and process it as an NPC dialogue request.	Passed
FT-04	LLM-Based NPC Response	The system should send the player input through the OpenRouter API using the Claude Haiku 4.5 model and display the generated NPC response in the game.	Passed
FT-05	Dialogue Flow Continuity	After receiving an NPC response, the interaction should continue without closing or breaking the dialogue system unexpectedly.	Passed
FT-06	Playable Demo Loop	The Unity demo should run as a playable proof-of-concept showing the core LLM-powered NPC interaction experience planned for the project.	Passed

Table 5. Functional Test Result Recording Table

3.2 Interface Tests

Test ID	Interface Area	Expected / Reported Verification Target	Result
IT-01	Dialogue User Interface	The dialogue panel should open correctly during NPC interaction and show the player input area clearly.	Passed
IT-02	Player Input Field	The text input field should allow the player to write and submit natural-language messages during conversation.	Passed
IT-03	NPC Response Display	The response generated by the LLM should appear correctly in the dialogue interface in a readable format.	Passed
IT-04	Unity-API Communication	The Unity client should successfully communicate with the OpenRouter API and receive responses from the Claude Haiku 4.5 model.	Passed
IT-05	Gameplay-to-Dialogue Transition	Switching from normal gameplay to dialogue interaction and returning to gameplay should work without causing interface conflicts.	Passed

Table 6. Interface Test Result Recording Table

3.3 Validation Tests

Test ID	Validation Focus	Expected / Reported Verification Target	Result
VT-01	Main Project Objective	The final project should demonstrate a working Unity-based life simulation demo with LLM-powered NPC interaction.	Passed
VT-02	Replacement of Static Dialogue	The implemented system should show that NPC replies can be generated dynamically instead of relying only on fixed, pre-written dialogue trees.	Passed
VT-03	Technical Feasibility of LLM Integration	The project should validate that an external LLM service can be integrated into a real-time game prototype and used during actual gameplay.	Passed
VT-04	Model and API Integration Choice	The OpenRouter API with Claude Haiku 4.5 should support the final playable demo and provide usable NPC responses during testing.	Passed
VT-05	Demo Readiness	The project should be stable enough to be shown as a working demonstration of the proposed system.	Passed

Table 7. Validation Test Result Recording Table

3.4 Test Assessment

A total of 16 test cases were evaluated across functional, interface, and validation testing categories. All planned test cases were completed successfully, and the obtained results confirmed that the implemented systems operate together as intended within the final prototype. The testing process verified the stability of the main gameplay and interaction flow, including player movement, NPC interaction, free-text dialogue input, asynchronous LLM-based response generation through the OpenRouter API, and the presentation of responses inside the Unity UI environment.

The completed tests also confirmed that the core communication pipeline between the Unity client and the external LLM service functions reliably during normal gameplay scenarios. Supporting gameplay systems such as inventory handling, day progression, relationship stat updates, and interaction-triggered UI feedback were also observed to operate consistently throughout the tested demo flow. In addition, fallback behavior and general gameplay continuity remained stable during typical interaction scenarios.

No major blocking issue was encountered during the final demonstration and testing stages. However, since the system depends on external API communication, response latency and third-party service availability remain important considerations for future development. Further improvements may include larger-scale stress testing, longer gameplay sessions, expanded NPC interaction scenarios, and more advanced testing of long-term dialogue consistency and gameplay balancing.

4- Tools and Technologies Used

LLM Valley was developed using Unity and C# together with external LLM services to support real-time NPC conversations. The project combines gameplay systems such as farming, inventory management, dialogue interaction, and local save/load functionality within a single playable prototype. OpenRouter API and the Claude Haiku 4.5 model were used for dynamic response generation, while local JSON-based persistence was used to manage game state and player progression.

Tool / Technology	Use in the Project
Unity 6.3 LTS	Game engine used to develop the 2D simulation environment, interaction systems, and UI behavior.
C#	Main programming language for Unity gameplay scripts and system components.
OpenRouter API (Claude Haiku 4.5)	Remote LLM service used in the final demo for dynamic NPC response generation.
HTTPS and JSON payloads	Communication format between the Unity client and the external LLM API.
UnityWebRequest	HTTP communication mechanism described for API request handling in the low-level design.
JsonUtility	Unity serialization utility used for local save/load data handling.
TextMeshPro / TMP_Text	Readable text rendering for dialogue history and UI labels, as listed in the low-level glossary.
GitHub and MS Teams	Collaboration and development coordination platforms described in the project proposal.
Prompt Engineering	Method used to shape NPC personas, response boundaries, and contextual dialogue behavior.

Table 8. Tools and Technologies Used

4.1 Use of Background and Reference Resources

Throughout the development process, both technical documentation and genre-related references were used to support implementation and design decisions. Unity's official documentation was frequently referenced for gameplay systems, UI implementation, scene management, and JSON-based persistence. Additional software engineering resources and object-oriented design principles were used while structuring the project architecture and subsystem organization.

Background research related to LLM-supported NPC interaction, prompt design, and context-aware dialogue systems also helped shape the overall direction of the project. In addition, life simulation and farming games such as Stardew Valley, Travellers Rest, and Necesse were examined as gameplay and interaction references during the design process.

5- Learned Lessons

The development process of LLM Valley showed that integrating a Large Language Model into a game environment involves much more than simply sending player text to an external API. The AI system must operate together with gameplay flow, UI feedback, input handling, game-state management, persistence systems, and overall pacing. This made the project valuable for understanding how AI-driven interaction becomes part of a larger software architecture rather than an isolated gameplay feature.

One of the most important lessons learned during development was the importance of modular architecture. Keeping the AI service layer separated from the core gameplay systems made the project easier to maintain and allowed the LLM provider to be changed without requiring major architectural modifications. The project also demonstrated how strongly response latency affects player experience. Even when response quality is acceptable, delays without proper UI feedback can make the gameplay feel unresponsive. Because of this, asynchronous communication and visible feedback states became essential parts of the implementation.

Another important observation was that fallback behavior is necessary in AI-connected applications. API failures or timeout situations should not interrupt the gameplay loop or cause the application to become unstable. Similarly, context management became one of the most challenging aspects of the dialogue system, since NPC personality, relationship history, current game state, and prompt size limitations all needed to remain balanced during interaction.

The project also showed that gameplay systems such as farming, trading, and progression mechanics help give meaning and context to AI-generated dialogue. Without a surrounding gameplay loop, the interaction system would behave more like a standalone chatbot than an integrated simulation experience. In addition, the development process highlighted the importance of ethical and professional considerations such as content safety, privacy, transparency of AI-generated responses, and reliable system behavior.

Finally, the project emphasized the importance of maintaining consistency between design documents, implementation decisions, testing procedures, and presentation materials throughout the development lifecycle. Keeping the final prototype aligned with the earlier architecture reports and testing plans was an important part of successfully completing the project.

6- Conclusion

LLM Valley successfully demonstrated the integration of Large Language Models into a real-time NPC interaction system within a playable Unity-based environment. The final implementation combined gameplay systems such as farming, inventory management, relationship progression, and dialogue interaction with an external LLM service through the OpenRouter API and the Claude Haiku 4.5 model. The completed prototype showed that dynamic, context-aware NPC conversations can operate together with traditional gameplay systems inside a responsive game architecture.

Throughout the project, important engineering challenges related to asynchronous communication, API reliability, gameplay continuity, UI responsiveness, and dialogue context management were explored and addressed. The development process also highlighted the importance of modular software architecture when integrating AI-supported systems into interactive applications.

The final implementation achieved the primary objectives defined at the beginning of the semester and validated the technical feasibility of using cloud-based LLM services for NPC interaction in a real-time gameplay environment. Although the project was developed as a proof-of-concept prototype with a limited gameplay scope, the resulting architecture provides a strong foundation for future improvements such as expanded NPC memory systems, more advanced world simulation, additional gameplay mechanics, and larger-scale interaction scenarios.

Overall, the project demonstrated that LLM-supported dialogue systems can contribute to more dynamic and flexible player interaction experiences while remaining compatible with modern game-development workflows and software-engineering practices.

7- REFERENCES

- [1] L. Song, "LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms," 2025.
- [2] L. M. Csepregi, "The Effect of Context-Aware LLM-Based NPC Conversations on Player Engagement in Role-Playing Video Games," Aalborg University, Copenhagen, Denmark, 2024.
- [3] G. Zeng, "A Review of AI-Based Game NPCs Research," Applied and Computational Engineering, vol. 15, pp. 155-159, 2023.
- [4] Unity Technologies, "Unity Manual and Scripting API," Unity Documentation, 2024. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [5] Unity Technologies, "Using JsonUtility for Saving and Loading Data," Unity Documentation, 2023. [Online]. Available: <https://docs.unity3d.com/ScriptReference/JsonUtility.html>
- [6] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns, and Java, 2nd ed. Prentice-Hall, 2004.
- [7] ConcernedApe, Stardew Valley [Video game]. ConcernedApe, 2016. [Online]. Available: https://store.steampowered.com/app/413150/Stardew_Valley/
- [8] S. Edwards, Travellers Rest [Video game]. Isolated Games, 2020. [Online]. Available: https://store.steampowered.com/app/1139980/Travellers_Rest/
- [9] Fair Games Studio, Necesse [Video game]. Fair Games Studio, 2019. [Online]. Available: <https://store.steampowered.com/app/1169040/Necesse/>